AZINHACK

FactFinder

AI-powered claim-level fact checking Using TinyFish

Verify Every Claim. Discover the Evidence.

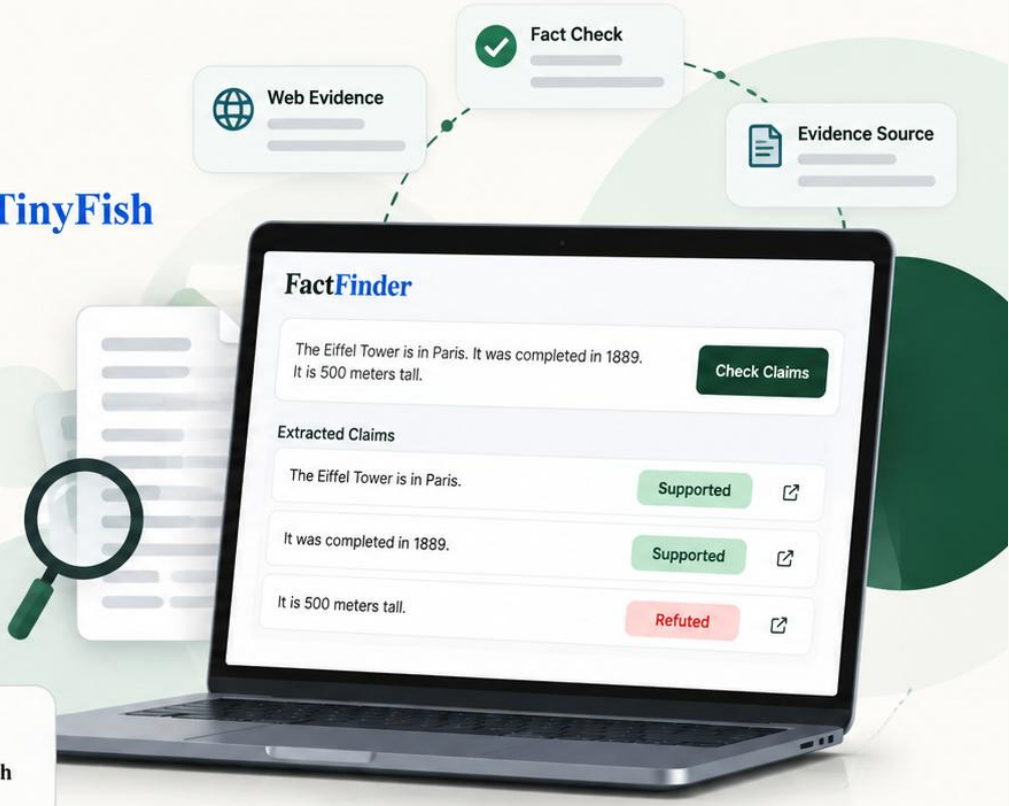
TEAM NAME
Fine Shyts

TEAM MEMBERS
Anurag Kumar, Dev Kumar, Kunal, Uday Singh

COLLEGE / ORGANIZATION
USICT, GGSIPU

TECHNOLOGY STACK

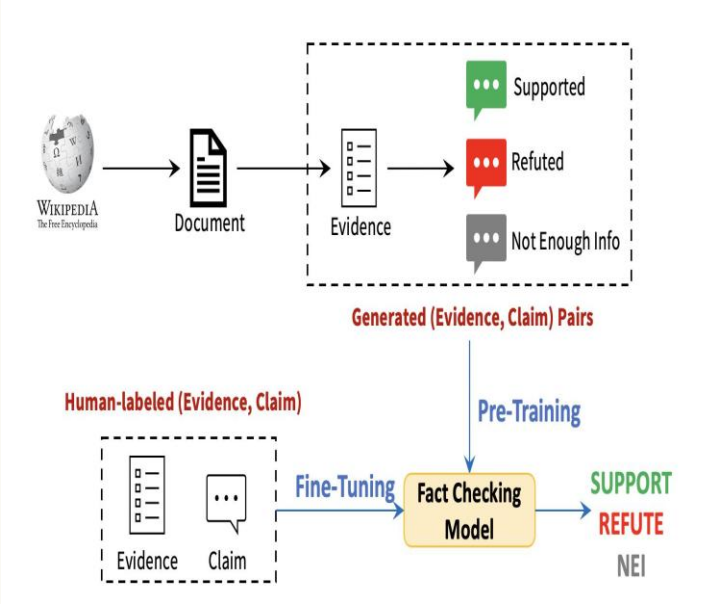
Python | LangGraph | Next.js | TinyFish



THE PROBLEM

One answer can contain many different truths.

- **Mixed information**: An AI-generated answer may combine correct facts with outdated, misleading, or incorrect claims. Each factual claim needs to be checked independently.
- **Hidden errors**: A single True/False label cannot show which statements are correct and which require correction.
- **Scattered evidence:** Finding reliable web sources and comparing them against individual claims takes time.
- **Manual verification:** Readers must research each statement themselves, making the process slow and difficult to scale.



THE SOLUTION : FactFinder breaks a statement into individual claims, searches for relevant web evidence using the project's verification workflow, and classifies each claim as Supported, Refuted, or Insufficient Evidence. It provides explanations and confidence levels to help users understand the results.

THE SOLUTION

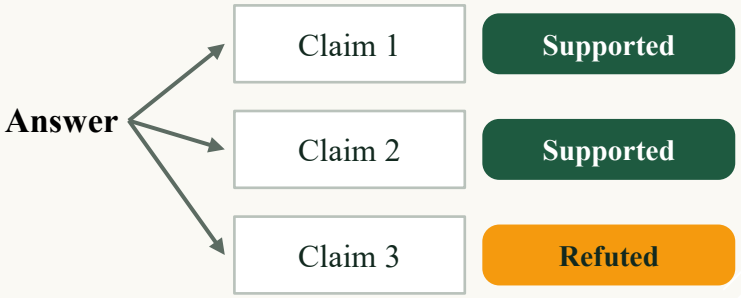
Check every claim, not just the answer.

TRADITIONAL CHECK

Whole answer → True / False?

One label for the whole answer. You cannot tell which sentence is wrong.

CLAIMAI



THREE POSSIBLE RESULTS

Supported

Reliable sources found online agree with the claim, so it can be kept as it is.

Refuted

Sources directly contradict the claim, so this is the exact statement that needs to be corrected.

Unresolved

Evidence conflicts or stays thin even after retries. Shown honestly as open, not guessed.

FLOW OF SOLUTION

Four stages turn an answer into an evidence trail.

1 SUBMIT

"The Eiffel Tower is in Paris. It was completed in 1889. It is 500 meters tall."

Paste a question together with its AI-generated answer, or any text you want checked, into the web interface. Nothing else is needed to start.

OUTPUT
Raw text

2 EXTRACT

1. The Eiffel Tower is in Paris.
2. It was completed in 1889.
3. It is 500 meters tall.

The Claim Extractor splits the text into individual sentences, removes opinions, fixes pronouns like "she" or "it", and rewrites every fact as a standalone, testable claim.

OUTPUT
List of atomic claims

3 VERIFY

Generate search queries for each claim

↓

Fetch relevant web sources

↓

Use LLM to judge and classify the claim

For each claim, an LLM writes search queries, retrieves web results and judges how well the sources support the claim. Unclear cases are retried.

OUTPUT
Verdict and sources

4 REPORT

The Eiffel Tower is in Paris.	Supported
It was completed in 1889.	Supported
It is 500 meters tall.	Refuted

The Fact Checker combines every result into one clear report that shows the verdict, the supporting sources and the reasoning behind each claim.

OUTPUT
Claim-level report

Every stage is separate, so its output can be inspected or edited before the next stage runs.

TECH STACK & APPROACH

Agents connect the interface to evidence.

01 ORCHESTRATION
LangGraph runs the Claim Extractor, Claim Verifier and Fact Checker agents as one coordinated workflow.

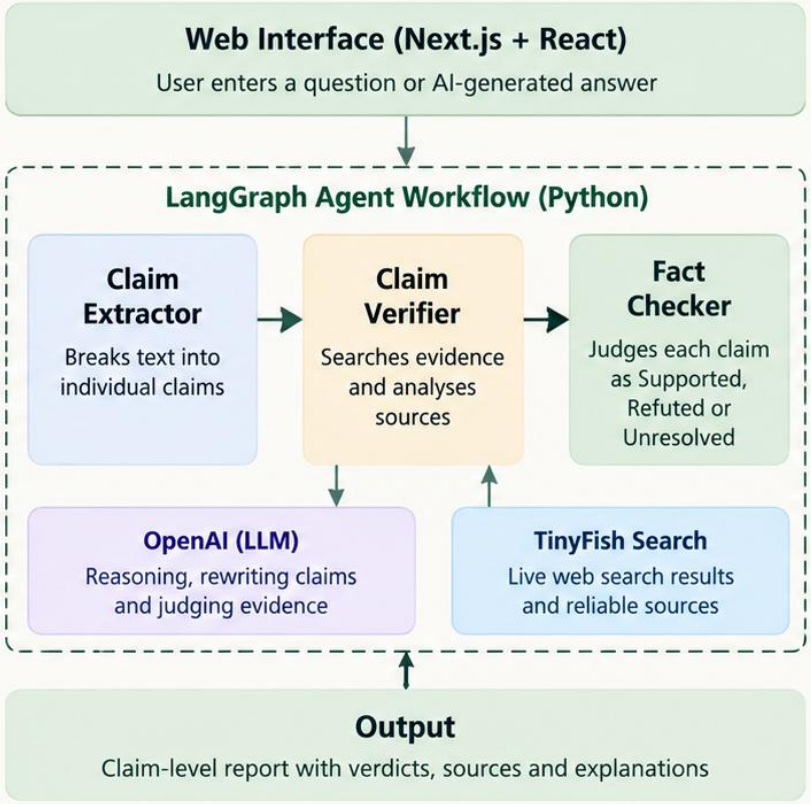
02 REASONING
OpenAI models understand the text, rewrite claims and judge whether the evidence supports them.

03 RETRIEVAL
TinyFish Search fetches live web results for each claim, so verdicts rest on current and reliable online evidence.

04 INTERFACE
A Next.js + React web app is the front end; the agent layer behind it runs in Python.

05 SETUP
pnpm + Turborepo manage the web app, Poetry manages the Python agents, and LangGraph runs the backend.

SYSTEM ARCHITECTURE



MODULE 1: CLAIM EXTRACTOR

Turning messy text into testable claims.

1 Split → 2 Select → 3 Disambiguate → 4 Decompose

1 Split

Break the text into sentences while keeping the surrounding context.

2 Select

Keep only sentences that contain a verifiable fact. Opinions and vague statements are dropped.

3 Disambiguate

Resolve pronouns and vague references. Several LLM votes settle unclear cases, or drop them.

4 Decompose

Rewrite each sentence into standalone, atomic claims that can be checked independently.

EXAMPLE

"Marie Curie won two Nobel Prizes, and she discovered polonium."

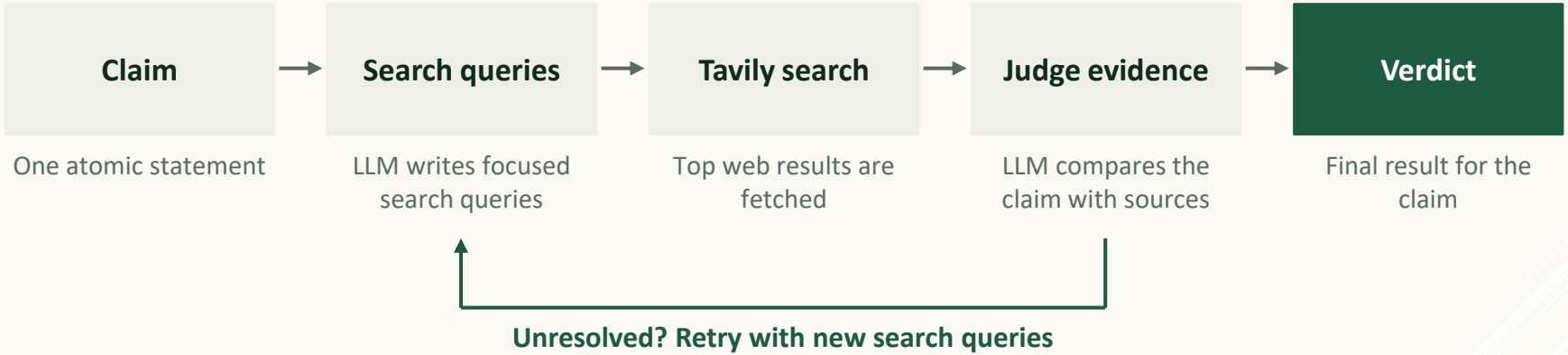
"she" becomes "Marie Curie"

- Marie Curie won two Nobel Prizes.
- Marie Curie discovered polonium.

Configurable in the config folder: LLM temperature, number of votes and more. The method is based on Claimify (Metropolitansky & Larson, 2025); the voting step is an addition by this project.

MODULE 2: CLAIM VERIFIER

Each claim is checked against live evidence.



Supported Sources agree with the claim.

Refuted Sources directly contradict it.

Unresolved Conflicting or thin evidence, even after retries.

Retry loop: if the evidence is unclear, the LLM writes new queries and searches again before deciding. Configurable: number of web results, retry attempts and LLM temperature.

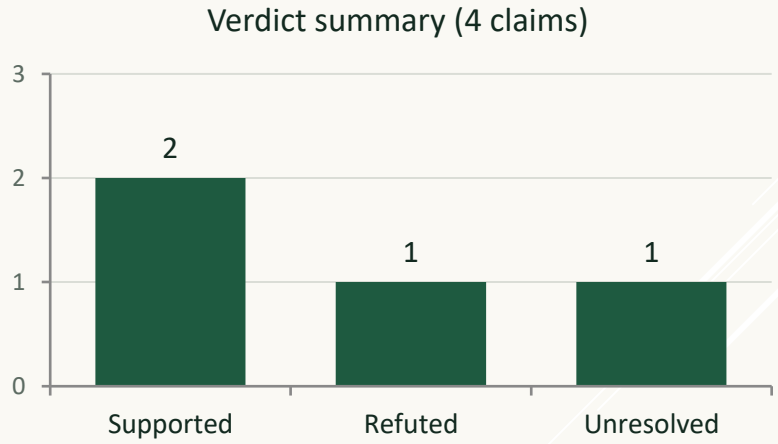
MODULE 3: FACT CHECKER

One report, with the evidence path for every claim.

The Fact Checker is the orchestrator. It calls the extractor, sends every claim to the verifier, and then builds the final report, so you see the overall picture and every individual claim in one place.

SAMPLE REPORT (ILLUSTRATIVE)

Eiffel Tower is in Paris.	Supported
Completed in 1889.	Supported
It is 500 meters tall.	Refuted
Claim with conflicting sources.	Unresolved



Evidence path: for every claim the report keeps the search queries, the retrieved sources and the reasoning behind the verdict, so any result can be traced and double-checked.

FEASIBILITY & COMPETITORS

ClaimAI makes uncertainty inspectable.

Capability	Generic chat	Basic search	ClaimAI
Primary output	An answer	A list of sources	**Claim-level results**
Checks each statement	No	No	**Yes**
Evidence path	Not necessarily structured	Retrieved sources only	**Linked to each claim**
Uncertainty	May be hidden	Needs interpretation	**Supported, refuted or unresolved**

WHY IT MATTERS

Claim-level

Each statement is judged on its own, so uncertainty stays attached to the exact claim that needs review.

Explainable

Extraction, retrieval, verification and reporting are separate stages, so each can be inspected.

Honest

Conflicting or insufficient evidence stays visible as unresolved instead of being forced into true or false.

Controllable

LLM voting and multi-retry search give ambiguous cases a path to a better answer.

FEASIBILITY, SETUP & RESEARCH

Feasible now, with a clear path ahead.

RUN IT LOCALLY

Install tools

Node 22+, pnpm 10, Python 3.11+, Poetry 2.1+ and the LangGraph CLI

→

Add API keys

Copy .env.example to .env and add OPENAI_API_KEY and TAVILY_API_KEY

→

pnpm setup:dev

Installs the web (pnpm) and Python (Poetry) packages in one command

→

pnpm dev

Starts the web app and the agents; open localhost:3000

1

NOW · Feasible today

- Local full-stack demo
- Web interface + agent workflow
- Open source (MIT)

2

NEXT · Evaluation agent

- Evaluation agent for overall performance
- Accuracy and reliability metrics

3

LATER · Public API service

- Public API for other apps, no local setup needed
- Robust hosted service

Research basis: Claimify (Metropolitansky & Larson, 2025) for claim extraction and SAFE (Wei et al., 2024) for evidence retrieval.